

Setting up the foundation: Dockerizing your application

The first critical step in enabling continuous deployment is to containerise your application — a process known as Dockerizing. This begins with writing a Dockerfile, a simple text file that contains a series of instructions Docker uses to build an image. For a basic Python Flask web application, the Dockerfile typically starts with a lightweight base image like *python:3.10-slim*, sets a working directory (commonly */app*), copies the project files from the host to the container, installs the required dependencies using *pip*, exposes a relevant port (such as 5000 for Flask), and finally defines the default command to run the application. Each of these steps ensures the application runs consistently in any environment, eliminating the “it works on my machine” problem. Once the Dockerfile is complete, the next step is building the Docker image locally using the command:

```
docker build -t <username>/<image-name>:<tag>
```

This command reads the Dockerfile and bundles the entire application, dependencies, and runtime environment into a single, reusable image. The image is tagged with a name and version to make it easier to track and manage. After a successful build, it's essential to verify that the image runs correctly. This is done by executing:

```
docker run -p <host-port>:<container-port> <image-name>
```

This launches a container based on the image and maps the internal port to a local machine port. Opening a browser and navigating to *localhost:<port>* confirms the app is running properly within the container. Once validated, the Docker image is ready to be shared. By logging into Docker Hub using the Docker login and

then pushing the image with:

```
docker push <username>/<image-name>:<tag>
```

...the image is uploaded to a central registry. Docker Hub acts as a version-controlled storage for images, making them accessible to deployment pipelines, cloud environments, or team members. This step not only simplifies the distribution of applications but also integrates seamlessly with CI/CD pipelines that can pull the image and deploy it automatically. Dockerizing your app, testing it locally, and publishing it to Docker Hub sets the foundation for scalable, consistent, and automated deployment — turning your app into a ready-to-ship unit that can move smoothly across the development, staging, and production stages with minimal friction.

Automating deployment with GitHub Actions

Automating deployment with GitHub Actions is a powerful way to streamline and standardise the continuous deployment (CD) process for your applications. It begins with creating a GitHub repository to host your application's source code. Within this repository, GitHub Actions uses workflow files defined in the *.github/workflows/* directory, typically with a *.yml* extension. The structure of a workflow starts with the *name*: field to assign a recognisable title, followed by the *on*: field which specifies the triggering events like push to the main branch. The *jobs*: section outlines the stages of the workflow, where each job contains multiple steps: that define individual actions such as checking out code or logging into Docker. Within steps, *uses*: allows integration with pre-built community actions, *env*: sets environment variables, and *secrets*: is used to securely reference sensitive credentials such as Docker Hub login or SSH keys. A complete GitHub Actions

workflow for CD may include checking out the code using *actions/checkout@v3*, logging into Docker Hub using secrets as follows:

```
docker login -u ${{ secrets.DOCKERHUB_USERNAME }} -p ${{ secrets.DOCKERHUB_TOKEN }}
```

...pulling the latest image, and deploying it by connecting to a remote server via SSH to stop the old container, remove it, and run the updated container using Docker Run.

To ensure these secrets are securely used during deployment, GitHub provides a ‘Secrets’ feature within each repository's settings. To configure secrets like *DOCKERHUB_USERNAME* and *DOCKERHUB_TOKEN*, go to your GitHub repository, click on *Settings* → *Secrets and variables* → *Actions*, and then click ‘New repository secret’, where you can add your Docker Hub credentials securely. This ensures that sensitive information does not appear in your workflow code and is protected during execution.

Implementing the continuous deployment workflow

To implement the CD workflow, start by making a small change to your application code. Open your project in a text editor or IDE such as VS Code and modify a visible element—for example, change the homepage message in *app.py* from ‘Hello, World!’ to ‘Hello from GitHub Actions!’. Once the change is made, save the file, and use the terminal to stage and commit the update with commands like *git add* . followed by *git commit -m “Updated homepage message for CD test”* . Push the committed change to the main branch using *git push origin main* . This push will automatically trigger your GitHub Actions workflow if it has been configured correctly.

After pushing, navigate to your repository on GitHub and click the